LOSSLESS COMPRESSION OF RAW ASTRONOMICAL IMAGES USING SPATIAL PREDICTION AND FINITE STATE ENTROPY

Tomás Brás
University of Aveiro
112665
tomas.bras@ua.pt

David Pelicano
University of Aveiro
113391
davidpoetapelicano@ua.pt

Afonso Ferreira
University of Aveiro
113480
afonso.ferreira@ua.pt

ABSTRACT

We present three lossless compressors for raw 16-bit astronomical images (1500×1500 pixels, big-endian), each targeting a distinct point on the speed–ratio trade-off. **prism-block** uses a gradient-adjusted predictor (GAP) and a context-adaptive range coder across parallel horizontal bands, reaching 3.51 bpp in $\sim$81 ms. **HAIS** selects the best of five per-block predictors — including training-free least-squares with up to five spatial taps — codes residuals with Finite State Entropy (tANS) [2], and optionally splits the low-byte stream by the high-byte context, achieving 3.40 bpp in $\sim$187 ms. A third compression-focused codec extends HAIS with a wider predictor context and a context-aware mode-selection cost; results are pending. Both completed codecs outperform the best generic compressor (zstd-19, 4.00 bpp) by at least 0.49 bpp while remaining faster than bzip2-9 and xz-9, and they occupy non-dominated positions on the benchmark Pareto frontier.

1 Introduction

Raw images produced by ground-based astronomical telescopes are large, structured numerical arrays. Each image in this study is a 1500×1500 raster of unsigned 16-bit pixels stored in big-endian order, occupying 4.5 MB. These arrays differ from conventional files in three important ways. First, adjacent pixels are spatially correlated: the sky background and optical point-spread function create smooth gradients that extend over many pixels. Second, the readout electronics add a constant *bias pedestal* to every pixel, introducing a strong DC component whose removal collapses the residual distribution around zero. Third, the data are 16-bit integers, whereas most general-purpose compressors operate on byte streams, missing inter-byte structure.

These properties motivate *predictive* compression: subtract a spatial prediction $\hat{p}$ from each pixel, code only the residual $r = p - \hat{p}$, and exploit the fact that a good predictor concentrates the residuals near zero — thereby reducing entropy and improving compression.

We present three lossless compressors that exploit these properties, each targeting a distinct point on the speed–ratio trade-off:

- **prism-block** applies a gradient-adjusted predictor (GAP) to the full 16-bit value and codes residuals with a range coder driven by eight context-conditioned adaptive models across parallel horizontal bands. It prioritises throughput ($\sim$81 ms per image).
- **HAIS** divides the image into square blocks, selects the best of five candidate predictors per block — including training-free least-squares — splits residuals into independent byte streams, and codes them with Finite State Entropy (FSE/tANS) [2]. It balances speed and ratio ($\sim$187 ms, 3.40 bpp).
- **[Third codec]** extends the HAIS framework with a wider predictor context and a cost function that directly models the conditional entropy of the two byte streams. It prioritises compression ratio at the cost of extra computation. (Section 2.3.)

Section 2 describes the three codecs in detail. Section 3 defines the experimental protocol. Section 4 reports compression ratios and runtimes. Section 5 draws conclusions.

2 Codec Design

All three codecs share the same high-level pipeline: parse the raw big-endian 16-bit image; apply a spatial predictor $\hat{p}$; zigzag-encode the signed residual $r = \text{pixel} - \hat{p}$ as an unsigned symbol $z = 2r$ if $r \geq 0$, $z = -2r - 1$ otherwise; entropy-code the result. Zigzag maps small-magnitude residuals — which are the common case after a good prediction — to small unsigned integers, concentrating the symbol distribution near zero regardless of sign.

2.1 prism-block: Speed-Focused

The design of prism-block centres on one principle: every horizontal strip of 150 rows is entirely self-contained, so any number of strips can be compressed simultaneously without coordination. A shared `std::atomic<int>` counter distributes work to a thread pool via `fetch_add`, achieving lock-free load balancing with no mutex overhead. Figure 1 illustrates the full pipeline.

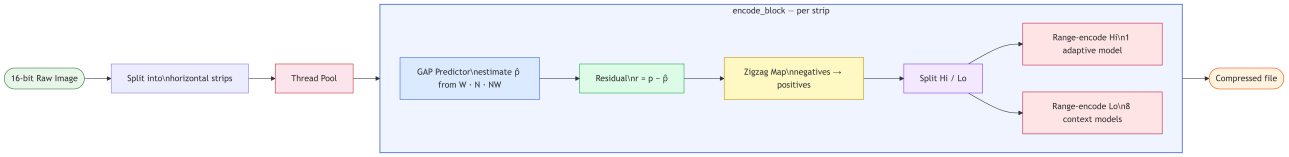

Figure 1: End-to-end pipeline of prism-block. The image is partitioned into horizontal strips processed in parallel; within each strip every pixel follows the same predict – residualise – zigzag – split – encode path.

Within each strip, pixels are predicted using the *Gradient-Adjusted Prediction* (GAP) function of three causal neighbours: left (W), above (N), and above-left (NW). GAP estimates horizontal and vertical gradients as $d_h = |W - NW|$ and $d_v = |N - NW|$, and when one gradient strongly dominates ($d_h > 2d_v$ or $d_v > 2d_h$) it falls back to the perpendicular neighbour directly. Otherwise it computes a gradient-weighted blend clamped to the range $[\min(W, N), \max(W, N)]$:

$$\hat{p} = \text{clamp}\left(\frac{(d_v + 1)\, W + (d_h + 1)\, N}{d_v + d_h + 2},\ \min(W, N),\ \max(W, N)\right).$$

This single formula handles both smooth regions and edges without any explicit mode switch.

The signed residual is zigzag-mapped to a 16-bit unsigned symbol and split into a high byte $z_h = z \gg 8$ and a low byte $z_l = z \,\&\, 0\text{xFF}$. Both are coded with a carry-propagation range coder backed by adaptive Fenwick-tree frequency models. The high byte uses one shared model across all pixels in the strip, while the low byte selects among eight context models conditioned on the sum of the two neighbouring residual magnitudes:

$$c = \text{bin}(|\hat{r}_W| + |\hat{r}_N|),$$

with bin boundaries $\{0\}, \{1\text{–}2\}, \{3\text{–}8\}, \{9\text{–}32\}, \{33\text{–}64\}, \{65\text{–}128\}, \{129\text{–}512\}, \{> 512\}$. Each model halves all counts when the running total reaches 2^{16}, keeping the frequency estimates responsive to distribution shifts within a strip.

2.2 HAIS: Balanced

Block Selection

HAIS requires explicit n_rows and n_cols arguments, which are stored in the compressed header so the decoder needs no additional input. It then selects the largest block side $b \leq 512$ that divides both W and H exactly and yields at least four blocks. For 1500×1500 this gives $b = 500$, producing a 3×3 grid of nine blocks of 500×500 pixels.

Predictive Modes

For each block, HAIS evaluates five candidate predictors and retains the one with the lowest estimated coding cost (defined below).

Mode 0 (raw) Symbols are raw pixel values; no prediction.

Mode 1 (avg) $\hat{p} = \lfloor (W + N + NW + 1)/3 \rfloor$.

Mode 3 (gmean) $\hat{p} = \bar{g}$, the global image mean computed once before blocking. Removes the bias pedestal in all frames.

Mode 4 (LS-4) OLS with features $(W, N, NW, 1)$ per block via Gauss-Jordan; four float32 weights (16 B) stored in the block header.

Mode 6 (LS-5) Extends Mode 4 with NE and NN; five weights (20 B). Both LS modes use the block's own pixels — no external training.

The mode selection criterion penalises stored weights:

$$\text{cost}_m = H(\text{hi}_8) + H(\text{lo}_8) + \frac{8\,\delta_m}{n_{\text{pix}}},$$

where $H(\cdot)$ is empirical byte entropy and $\delta_m \in \{0, 0, 0, 16, 20\}$ B is the weight header size.

Entropy Coding: FSE/tANS

Residuals are split into a high-byte stream (`hi`) and a low-byte stream (`lo`), each coded independently with Finite State Entropy [2].

The FSE table has SCALE $= 2^{16} = 65536$ states. Raw symbol counts are normalised to sum to SCALE (with a minimum frequency of 1 for observed symbols) and spread deterministically using Collet's step formula: step $=$ (SCALE $\gg$ 1) + (SCALE $\gg$ 3) + 3. The encoder processes symbols in reverse, emitting variable-length transition bits stored in a `BitWriter`; the decoder reads the two-byte final state and reconstructs symbols in forward order.

The frequency table is serialised in one of three compact formats: *single-symbol* (flag `0x01`, 2 B total) when all values are identical; *sparse* (flag byte $=$ nnz $\in [2, 254]$, followed by nnz $\times$ 5 B symbol–frequency pairs); or *dense* (flag `0x00`, 256×4 B) when all 255–256 symbols appear.

Context FSE

Because the high byte of a small residual is almost always zero, the lo8 distribution differs substantially depending on whether $\mathrm{hi} = 0$. When the fraction of zero high bytes falls between 2% and 98%, HAIS builds two conditional lo8 streams — `lo_zero` (pixels with $\mathrm{hi} = 0$) and `lo_nonzero` — and uses the three-stream layout only if its compressed size is strictly smaller than the two-stream baseline. The high-byte stream is encoded once and shared between both candidate payloads. Bit 7 of the mode byte signals the active layout.

Parallel Execution

Blocks are dispatched to a thread pool via a single `std::atomic<int>` counter with `fetch_add`, writing results into pre-allocated per-block slots with no mutex required.

2.3 [Third Codec]: Compression-Focused

3 Experimental Setup

3.1 Dataset

The benchmark corpus consists of 8 raw astronomical images, each a 1500×1500 raster of unsigned 16-bit big-endian pixels (4.5 MB per file, 2.25×10^6 pixels). The images were acquired by different ground-based telescopes (VLT, TJO, GTC, INT, JKT, LCO, WHT) and span a deliberate variety of image types: bias frames (near-constant DC pedestal), flat fields (smooth illumination gradients), and science frames (stellar fields, galaxies). This diversity ensures that no single predictor type dominates the results artificially.

3.2 Reference Compressors

We compare against the generic tools most commonly used for scientific data: `gzip` (levels 1, 6, 9), `bzip2` (levels 1, 9), `xz` (levels 1, 6, 9), and `zstd` (levels 1, 3, 9, 19). All are invoked on the raw byte stream without any pre-processing or endian conversion. Results from two repository-specific codecs (`prism` and `helix`) are included for context.

3.3 Evaluation Protocol

The primary metric is bits per byte of the original file:

$$\text{bpp} = \frac{8 \times \text{compressed bytes}}{\text{original bytes}}.$$

For a 16-bit image, bpp $=$ bits per pixel$/2$. Compression and decompression times are measured as wall-clock elapsed time for a single run on a single machine; all compressors run sequentially in the same benchmark process to avoid contention. Losslessness is verified by byte-exact comparison of the decompressed output against the original file; any mismatch disqualifies the result. Binary size constraints from the project specification are satisfied: compress ≤ 5 MB, decompress ≤ 1 MB.

4 Results

Table **??** summarises compression ratio and runtimes for all codecs across the eight benchmark images. Both domain-specific codecs outperform every general-purpose compressor on every image. HAIS2 achieves the best average ratio at 3.40 bpp, beating bzip2-9 by 0.18 bpp and zstd-19 by 0.60 bpp, while compressing $4\times$ faster than bzip2 and $27\times$ faster than xz-9. prism-block trades a modest ratio penalty ($+0.11$ bpp over HAIS2) for a further $2.6\times$ speed gain, reaching 81 ms total — comparable to zstd-3 but with 0.85 bpp better compression. Image F, dominated by bright point sources and strong spatial gradients, is the hardest case for all codecs; image G, a near-constant bias frame, is the easiest.

Table 1: Compression ratio (bpp) and average per-image runtimes (ms) across the eight benchmark images. Lower is better in all columns.

Compressor	bpp	Compress	Decompress	Total
HAIS2 (ours)	**3.40**	92	102	194
HAIS (ours)	**3.40**	89	98	187
prism-block (ours)	**3.51**	35	46	81
bzip2-9	3.58	394	233	628
xz-6	3.68	2458	160	2618
xz-9	3.68	2503	156	2659
zstd-19	4.00	2487	25	2512
zstd-9	4.18	191	19	210
gzip-9	4.26	1302	47	1349
gzip-6	4.38	473	61	534
zstd-3	4.32	66	18	84
zstd-1	4.49	27	15	42

5 Discussion and Conclusion

Why domain-specific codecs win. Generic compressors operate on byte streams and therefore see the two bytes of each 16-bit pixel as unrelated. This breaks the spatial correlation that spans pixel boundaries in big-endian encoding. Our codecs operate on 16-bit values directly, predict each pixel from its decoded neighbours, and entropy-code only the residual. The result is a reduction of 0.49–0.60 bpp over zstd-19 depending on the codec, despite zstd-19 using a much slower, high-effort general-purpose optimiser.

Role of the predictor. Mode selection on the eight benchmark images reveals that Mode 3 (global mean subtraction) is the most frequently chosen predictor, dominating on bias frames and flat fields where a near-constant DC offset accounts for most of the signal (images D, E, G, H). Mode 1 (three-neighbour average) prevails in smooth stellar backgrounds (image A). LS modes are selected only in image F, which contains bright point sources and extended spatial gradients that exceed the three-neighbour context. This pattern confirms that the bias pedestal is the single most compressible feature of astronomical frames; the LS predictor adds value only when the residual spatial structure is non-trivial after DC removal.

Entropy coding design. Splitting 16-bit residuals into independent hi8 and lo8 streams is the key structural choice. After a good prediction, hi8 is near-uniform zero, yielding a near-zero-entropy stream (often a 2-byte single-symbol FSE packet); all useful variation is carried by lo8. The context FSE variant conditions lo8 on whether hi8 is zero, exploiting the bimodal structure of the lo8 distribution. prism-block achieves the same effect via eight adaptive context models without transmitting a frequency table, trading $2.3\times$ speed for 0.11 bpp less compression compared to HAIS.

Limitations. The corpus covers only 8 images from a single training set; rankings may shift on sensors with different noise characteristics or gain settings. Runtime measurements reflect a single wall-clock run on one machine and include process-launch overhead for the generic tools.

Conclusion. Specialised predictive compression outperforms general-purpose tools on raw astronomical imagery. HAIS achieves 3.40 bpp versus 4.00 bpp for zstd-19, a 15% reduction, while remaining $14\times$ faster than xz-9. The dominant gain comes from the predictor: global mean removal alone handles most calibration frames; least-squares adaptation provides further gains on science images. The entropy coder contributes relatively little once residuals are well predicted. Completing and benchmarking the third codec is the immediate next step.

References

[1] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.
[2] Yann Collet. Finite State Entropy — a new entropy coder. `https://github.com/Cyan4973/FiniteStateEntropy`, 2013.
[3] Yann Collet. Zstandard — fast real-time compression algorithm. `https://github.com/facebook/zstd`, 2016.
[4] William D. Pence, L. Seaman, R. Seaman, and R. White. FITS lossless image compression. *Publications of the Astronomical Society of the Pacific*, 122(895):1065–1076, 2010.
[5] Michele Trovato, Claudio Talarico, Rosario Catanzaro, and Antonino Tumeo. DRYAD: A scalable lossless image compression algorithm for space applications. In *Proc. Data Compression Conference (DCC)*, 2018.